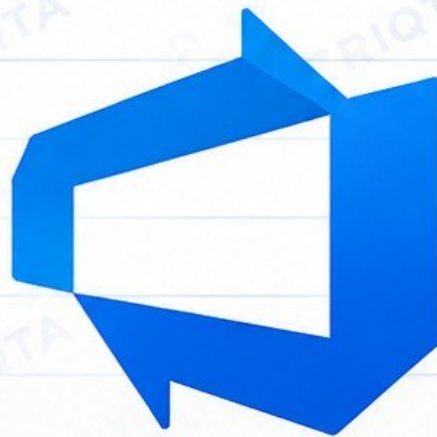


Azure DevOps

Pocket Guide



Azure
Boards



Azure
Repos



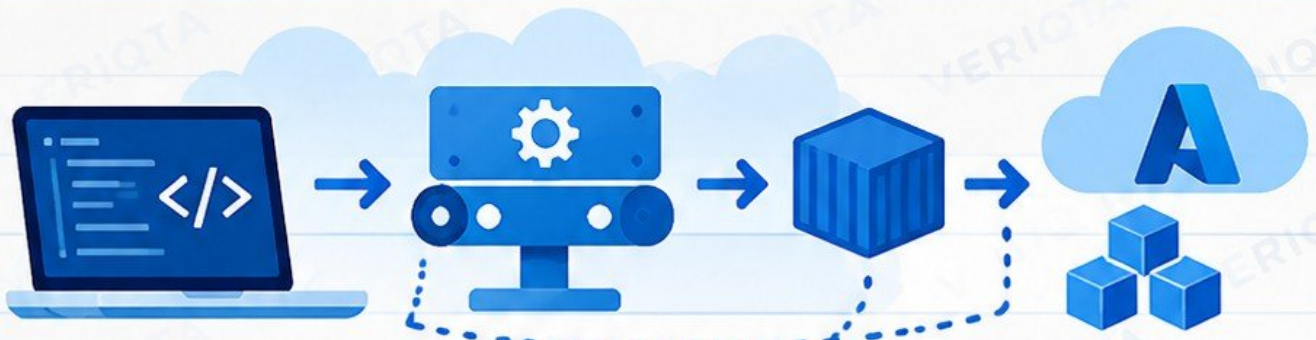
Azure
Pipelines



Test
Plans



Azure
Artifacts



Plan • Build • Test • Deploy • Deliver

Azure DevOps, The Big Picture

1. What is Azure DevOps?

Azure DevOps is a cloud platform from **Microsoft** that helps teams plan, develop, test, and deploy software. It provides tools for collaboration, source control, CI/CD, testing, and package management in one place.

2. Why Do Teams Use It?

- ✓ Improve team collaboration
- ✓ Track and manage work clearly
- ✓ Keep source code in a secure Git repository
- ✓ Automate builds, tests, and deployments
- ✓ Release software faster and more reliably
- ✓ Maintain quality and traceability
- ✓ Integrate with Azure and other tools

3. Main Services in Azure DevOps



Azure Boards

- Plan and track work
- Epics, stories, tasks
- Backlogs and sprints
- Keep the team aligned



Azure Repos

- Git repositories
- Source code management
- Branching and pull requests
- Code collaboration



Azure Pipelines

- Build, test, and deploy
- CI/CD automation
- YAML pipelines
- Deploy to Azure, on-prem or other clouds



Test Plans

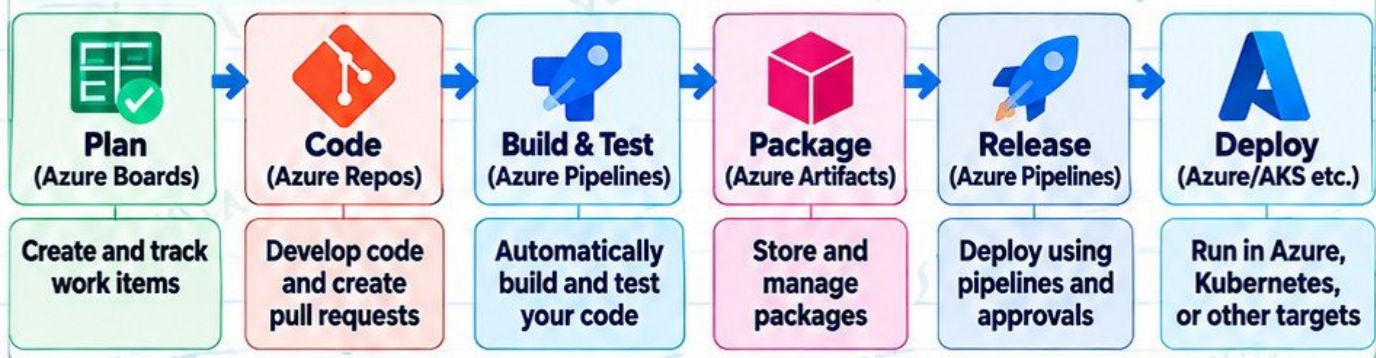
- Plan test cases
- Manual and automated testing
- Track test results
- Ensure software quality



Azure Artifacts

- Package management
- NuGet, npm, Maven, Python packages
- Share packages across teams
- Use in your pipelines

4. How It All Fits Together



5. Real World Example

A developer creates a new feature and pushes code to **Azure Repos**. **Azure Pipelines** automatically builds and tests the code. If successful, it creates a package and deploys it to a test environment. After approval, the same package is deployed to production.

6. Key Takeaways

- ✓ Azure DevOps brings the entire software delivery process into one platform.
- ✓ Each service has a specific role, but they work together.
- ✓ It helps teams deliver software faster, safer, and with better quality.
- ✓ You can use it with Azure, on-premises, or other cloud providers.

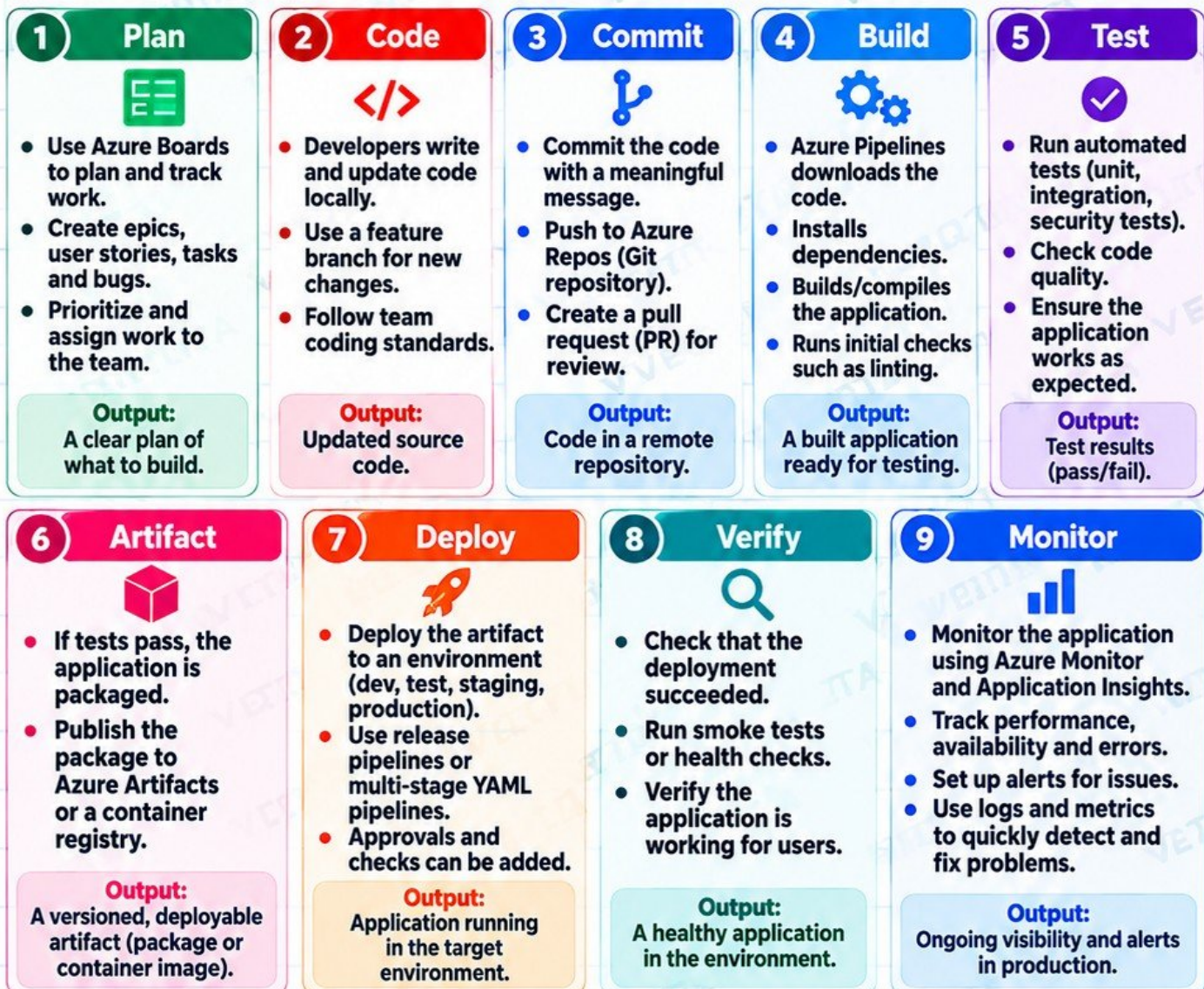
From Code to Production

1. The Big Picture

This is the complete flow from a developer writing code to the application running in production. **Azure DevOps** helps automate and manage this entire process.



2. What Happens at Each Stage?



3. Why This Flow Matters

- ✓ Delivers software faster and more reliably.
- ✓ Reduces manual work and human error.
- ✓ Improves code quality with automated testing.
- ✓ Provides visibility and traceability.
- ✓ Makes it easier to roll back if there is an issue.
- ✓ Helps teams work together efficiently.

Azure DevOps Organizations & Projects

1. What is Azure DevOps?

Azure DevOps is a cloud platform from **Microsoft** that provides tools for planning, developing, testing, and deploying software. It helps teams work together throughout the entire software development lifecycle.

2. Key Services



Azure Boards



Azure Repos



Azure Pipelines



Test Plans



Azure Artifacts

3. Organization vs Project

Organization



- Top-level container for all your work
- Has a unique URL

Example:

<https://dev.azure.com/yourorg>

Project



- A workspace inside an organization
- Used to separate work by team, product, or application

Example:

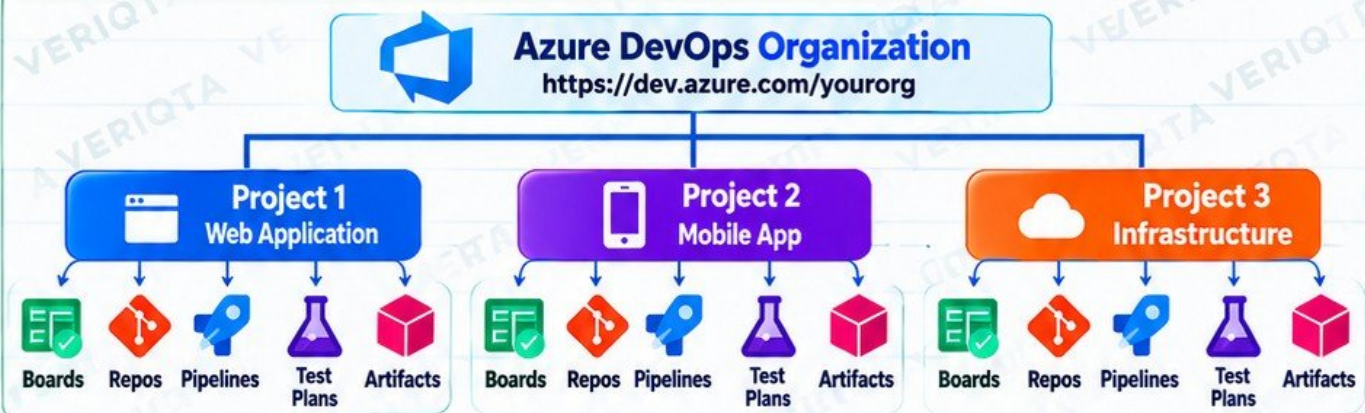
E-commerce Project

4. Teams



- Teams are groups of people who work on the same project.
- You can assign permissions to teams.
- Common teams: Developers, QA, DevOps, Project Managers.
- Teams help manage access and collaboration.

5. How a Real Azure DevOps Workspace is Structured



6. Repositories (Repos)



- Store your source code using Git.
- Each project can have multiple repositories.
- Used for version control, branches, and pull requests.

Example:

webapp-repo
infra-repo

7. Pipelines



- Automate build, test, and deployment of your code.
- Each project can have multiple pipelines.
- Pipelines can deploy to Azure, on-premises, or other clouds.

Example:

Build Pipeline
Release Pipeline

8. Permissions



- Control who can access and what they can do.
- Permissions can be set at the organization, project, or resource level (Repos, Pipelines, etc.).
- Use teams and roles to manage access.

Example Roles:

Reader, Contributor, Project Admin, Organization Owner

9. Key Takeaways

- ✓ An organization can contain multiple projects.
- ✓ Each project has its own Boards, Repos, Pipelines, Test Plans, and Artifacts.
- ✓ Teams help with collaboration and access management.
- ✓ Repositories store your code.
- ✓ Pipelines automate CI/CD.
- ✓ Permissions control what people can do.
- ✓ This structure keeps work organized and secure.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Azure Boards

1. What is Azure Boards?

Azure Boards is a work management tool in Azure DevOps. It helps teams plan, track, and manage work from an idea to a finished feature. It gives visibility to everyone and keeps the team aligned.

2. Work Item Types



Epic
Large goal or initiative



Feature
Major functionality



User Story
User requirement from the customer



Task
Work item to complete
A task

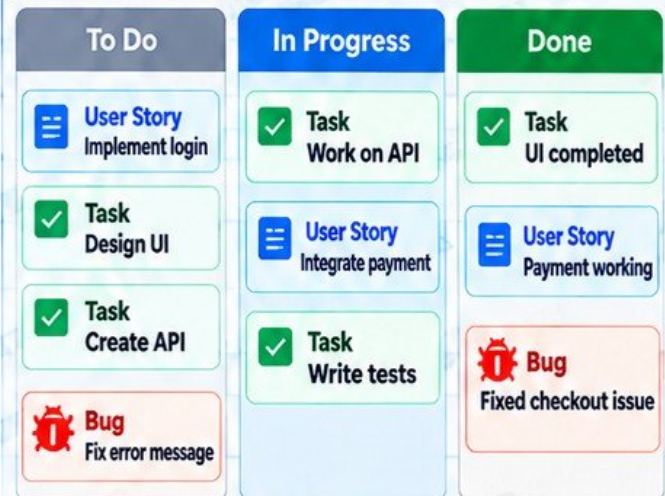


Bug
A defect that needs to be fixed

3. How Work Items Relate



4. Sprint Board Example



5. Backlogs

- **Product Backlog:** All work items, prioritized by business value.
- **Sprint Backlog:** Items selected for the current sprint.
- Teams can add, prioritize, and refine items regularly.

6. Work Flow from Idea to Implementation



7. Real World Example

- A company wants to add online payment.
- **Epic:** Improve customer experience
 - **Feature:** Add online payment
 - **User Story:** As a customer, I want to pay with a credit card
 - **Tasks:** Design UI, Create API, Write tests
 - **Bug:** Fix payment failure for some cards

8. Key Takeaways

- ✓ Azure Boards helps you manage the entire development lifecycle.
- ✓ Use Epics, Features, User Stories, Tasks, and Bugs to organize work.
- ✓ Backlogs keep work prioritized.
- ✓ Sprint boards show real-time progress.
- ✓ Everyone on the team gets visibility.
- ✓ It connects planning with actual implementation.

Azure Repos & Git Workflow

1. What is Azure Repos?



- **Azure Repos** is a cloud-based Git repository service in Azure DevOps.
- It stores your source code, tracks changes, and enables team collaboration.
- It supports Git (most common) and TFVC (legacy).
- You can work from your local machine using Git commands.

2. Git Workflow (High Level)



3. Clone a Repository

```
git clone https://dev.azure.com/yourorg/YourProject/_git/your-repo
```

- Copies the remote repository to your local machine.
- You only need to do this once.

4. Create and Switch to a Branch

```
git checkout -b feature/login-page
```

- Creates a new branch and switches to it.
- Use descriptive branch names.

Common Branch Types

- **feature/** – new feature
- **bugfix/** – fix a bug
- **hotfix/** – urgent fix
- **release/** – prepare a release

5. Make Changes and Commit

```
git status
git add .
git commit -m "Add login page"
```

- **git status** → shows changed files.
- **git add .** → stages changes.
- **git commit -m "message"** → saves changes locally.

6. Push to Azure Repos

```
git push origin feature/login-page
```

- Uploads your local branch to Azure Repos.
- The first time, the branch is created remotely.

7. Create a Pull Request (PR)

- Open your branch in Azure Repos.
- Click **New Pull Request**.
- Select source branch and target branch (e.g. **main**).
- Add a title and description.
- Request reviewers.

8. Review and Merge

- Team reviews the code.
- Address any comments.
- Once approved, merge the PR.
- The changes are now in the target branch (e.g. **main**).

9. Useful Git Commands

Command	What it does
<code>git clone <url></code>	Clone a repository
<code>git status</code>	Show changed files
<code>git add .</code>	Stage changes
<code>git commit -m "msg"</code>	Commit changes
<code>git push origin <branch></code>	Push to Azure Repos
<code>git pull origin <branch></code>	Get latest changes
<code>git branch</code>	List branches
<code>git checkout <branch></code>	Switch branch
<code>git log --oneline</code>	Show commit history

10. Real Project Example

-  Clone the repo
-  Create branch: feature/payment
-  Edit code (add payment feature)
-  Commit: Add payment feature
-  Push to Azure Repos
-  Create Pull Request
-  Team review
-  Merge to **main** branch

11. Beginner Mistakes

- ✗ Committing directly to **main**.
- ✗ Using unclear branch names.
- ✗ Forgetting to pull latest changes.
- ✗ Pushing sensitive files or credentials.
- ✗ Ignoring code review comments.
- ✗ Not writing clear commit messages.

12. Key Takeaways

- ✓ Use branches for your work.
- ✓ Commit small, meaningful changes.
- ✓ Push your branch to Azure Repos.
- ✓ Use pull requests for code review.
- ✓ Never commit directly to **main**.
- ✓ Keep your repository clean and secure.



Branches, Pull Requests & Protection

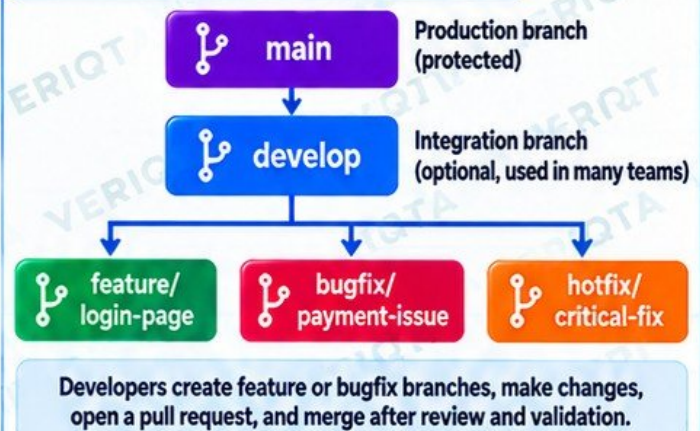
1. Why Not Commit Directly to Production?

- ❌ High risk of breaking the application.
- ❌ No code review.
- ❌ No automated checks (build, tests, security).
- ❌ Hard to track who made changes.
- ❌ Prevents collaboration.
- ❌ Can cause downtime and data loss.

Best Practice:

Never commit directly to the production branch (e.g. **main**). Always use branches and pull requests.

2. Common Branching Strategy



3. Create and Work on a Branch

```

# Switch to develop or main first
git checkout develop

# Create a new branch
git checkout -b feature/login-page

# Make your changes, then commit
git add .
git commit -m "Add login page"

# Push the branch to Azure Repos
git push origin feature/login-page
  
```

4. Pull Request (PR) Process

- 1. Push your branch**
Push the branch to Azure Repos.
- 2. Create Pull Request**
Select source and target branch (e.g. feature → main).
- 3. Add description**
Explain what changes were made and why.
- 4. Request reviewers**
Add team members to review.
- 5. Complete required checks**
Build, tests, and other policies must pass.
- 6. Merge**
After approval and successful checks, merge the PR.

5. Branch Policies (Protection)

Branch policies help protect important branches like main or production.

Common Policies:

- ✓ Require pull requests for changes
- ✓ Require a minimum number of reviewers
- ✓ Require build validation (CI pipeline)
- ✓ Require passing status checks (tests, security scans)
- ✓ Block direct pushes
- ✓ Require up-to-date branch before merging
- ✓ Limit who can approve

6. Build Validation in PR



7. Merge Conflict

- ⚡ Happens when the same part of a file is changed in different branches.
- Git cannot automatically decide which change to keep.
- You must manually resolve the conflict.

How to fix:

- Pull the latest changes
- Edit the conflicted files
- Remove conflict markers
- Keep the correct changes
- Commit and push again

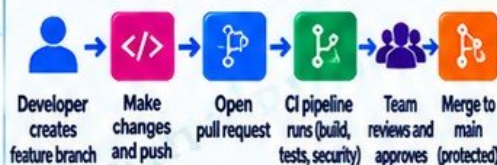
8. Resolve Merge Conflict (Example)

```

# Pull latest changes from main
git checkout feature/login-page
git pull origin main

# Edit conflicted files, then:
git add <file>
git commit -m "Resolve merge conflict"
git push origin feature/login-page
  
```

9. Protected Workflow Example



10. Key Takeaways

- ✓ Use branches for all changes.
- ✓ Use pull requests for code review.
- ✓ Enable branch policies on main.
- ✓ Use build validation and required checks.
- ✓ Resolve merge conflicts carefully.
- ✓ Keep the main branch stable and protected.



Your First Azure Pipeline

1. What is an Azure Pipeline?



- An Azure Pipeline is an automated CI/CD workflow in Azure DevOps.
- It builds, tests, and can deploy your application whenever code changes are pushed.
- It helps teams deliver code faster, safer, and more reliably.

2. What Happens After You Push Code?



1. Trigger

You push code to a branch (e.g. main).



2. Agent

A build agent picks up the job.



3. Steps

Runs the configured steps.



4. Build

Compiles your application.



5. Test

Runs automated tests.



6. Result

Shows success or failure.

3. Pipeline Trigger Types



CI Trigger

- Runs automatically when code is pushed to a branch.
- Common for main and feature branches.



PR Trigger

- Runs when a pull request is created or updated.
- Used to build and test changes before merging.



Manual Trigger

- Run the pipeline manually when needed.
- Useful for releases or testing.

4. Pipeline Flow Diagram



Developer Pushes Code



Azure Repos (Code Change)



Azure Pipeline (Triggered)



Agent (Hosted/Self)



Build (Compile)



Test (Automated Tests)



Publish (Artifacts)



Result (Success/Fail)

5. Simple Pipeline Example (azure-pipelines.yml)

```
trigger:
- main

pool:
vmImage: 'ubuntu-latest'

steps:
- script: echo "Building the application...."
  displayName: 'Build'
- script: echo "Running tests..."
  displayName: 'Test'
- script: echo "Build and test completed!"
  displayName: 'Result'
```

What Each Part Does

- **trigger:** Tells Azure DevOps when to run the pipeline.
- **pool:** Defines the build agent (Microsoft-hosted in this example).
- **steps:** List of tasks to run.
- **script:** Runs a command.
- **displayName:** Name shown in the UI.

6. Pipeline Run Example (UI)

Jobs in run #20250927.1		
YourProject		
✓ Job		1m 24s
✓ Initialize job		3s
✓ Build		45s
✓ Test		28s
✓ Publish artifacts		12s
✓ Post-job cleanup		3s

7. Common Tasks You Can Add



Build a .NET, Java, Python, or Node.js app



Run unit and integration tests



Publish build artifacts



Build and push Docker images



Deploy to Azure (App Service, AKS, etc.)



Run security scans (e.g. SAST)

8. Hosted vs Self-Hosted Agents

Microsoft-Hosted Agent



- ✓ Managed by Microsoft
- ✓ Ready to use
- ✓ Common tools pre-installed
- ✓ Good for most projects
- ✗ Does not have access to your internal network

Self-Hosted Agent



- ✓ You manage the machine
- ✓ Install your own tools
- ✓ Can access internal resources
- ✓ More control and customization
- ✗ You are responsible for maintenance and security

9. Key Takeaways

- ✓ A pipeline runs automatically after code changes.
- ✓ It uses an agent to execute steps.
- ✓ It builds, tests, and can publish or deploy your application.
- ✓ You can configure different triggers (CI, PR, manual).
- ✓ Pipelines help you deliver better software, faster and more reliably.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Understanding azure-pipelines.yml

1. Complete Example (azure-pipelines.yml)

```

1 trigger:
2   branches:
3     include:
4       - main
5
6 pool:
7   vmImage: 'ubuntu-latest'
8
9 variables:
10  appName: 'my-app'
11  buildConfiguration: 'Release'
12
13 stages:
14   - stage: Build
15     displayName: 'Build and Test'
16     jobs:
17       - job: BuildJob
18         displayName: 'Build Job'
19         steps:
20           - task: UseDotNet@2
21             displayName: 'Install .NET SDK'
22             inputs:
23               packageType: 'sdk'
24               version: '8.x'
25
26           - script: dotnet build --configuration $(buildConfiguration)
27             displayName: 'Build Project'
28
29           - script: dotnet test --configuration $(buildConfiguration)
30             displayName: 'Run Tests'

```

2. YAML Sections Explained



trigger

- Tells Azure DevOps when to run the pipeline.
- In this example, it runs when there are changes pushed to the main branch.



pool

- Specifies the build agent to use.
- Here we use a Microsoft-hosted agent with the Ubuntu latest image.



variables

- Defines reusable values.
- Here we define appName and buildConfiguration.
- You can use variables in tasks and scripts.



stages

- Groups jobs into stages.
- This example has one stage called Build.
- In real pipelines, you can have multiple stages (e.g. Build, Test, Deploy).



jobs

- A stage contains one or more jobs.
- This example has one job called BuildJob.
- Each job runs on an agent and contains steps.



steps

- A job contains a sequence of steps.
- Steps run tasks or scripts in order.
- Each step performs a specific action.



task

- A task is a pre-built action provided by Azure DevOps.
- Here we use UseDotNet@2 to install the .NET SDK.
- Tasks have inputs to configure their behavior.



script

- Runs a command in the agent's shell.
- Here we use dotnet build and dotnet test.
- You can run any CLI command using a script step.

3. Pipeline Structure (Visual View)



4. Key Syntax and Notes

- ✓ YAML is indentation-based.
- ✓ Use clear and consistent indentation.
- ✓ You can have multiple stages, jobs, and steps.
- ✓ Use variables to avoid hardcoding values.
- ✓ Combine tasks and scripts to automate your workflow.
- ✓ This is a basic example. Real pipelines can include deployments, environments, approvals, and more.

5. Common Agent Images

Image	Use Case
ubuntu-latest	Linux builds (most common)
windows-latest	Windows builds (.NET, etc.)
macos-latest	macOS builds (iOS, etc.)
self-hosted	Your own agent machine

6. Useful Variables (Examples)

Variable	Example
\$(Build.SourceBranch)	refs/heads/main
\$(Build.BuildId)	20250927.1
\$(Build.SourcesDirectory)	/agent/_work/1/s
\$(Build.ArtifactStagingDirectory)	/agent/_work/1/a
\$(BuildConfiguration)	Release
\$(appName)	my-app

7. Key Takeaways

- ✓ trigger decides when the pipeline runs.
- ✓ pool defines the build agent.
- ✓ variables store reusable values.
- ✓ stages organize the pipeline.
- ✓ jobs run a set of steps.
- ✓ steps can be tasks or scripts.
- ✓ tasks are built-in actions.
- ✓ scripts run any CLI command.



@veriqta_academy



@veriqta



@veriqta



@veriqta



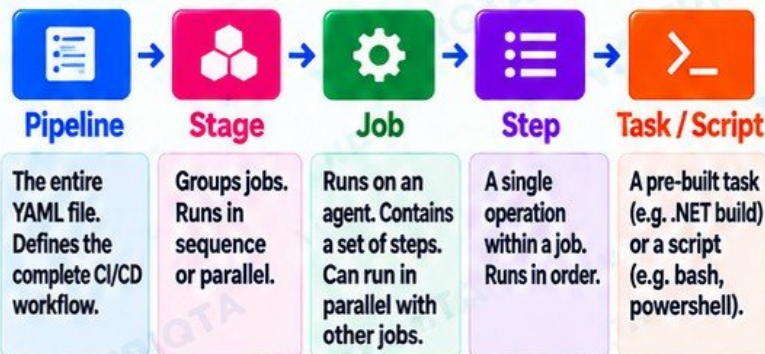
@veriqta



@veriqta

Stages, Jobs, Steps & Tasks

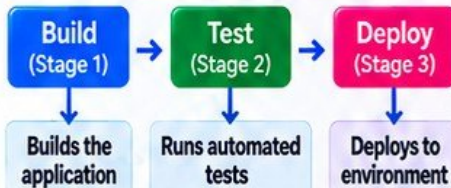
1. The Execution Hierarchy



3. How It Runs (Execution Flow)



4. Stages and Dependencies



Stage 2 runs only if Stage 1 succeeds.
Stage 3 runs only if Stage 2 succeeds.

Common Conditions

- ✓ **succeeded()** – runs if previous stage/job succeeded.
- ✗ **failed()** – runs if previous stage/job failed.
- ✗ **always()** – runs regardless of previous result.
- ✓ **succeeded('Build')** – runs if a specific stage succeeds.

5. Jobs: Types and Behavior

Agent Job (most common)



- Runs on a Microsoft-hosted or self-hosted agent.
- Used for build, test, deploy, etc.
- Each job gets a fresh agent machine.

Server Job



- Runs on Azure DevOps server (without an agent).
- Used for simple tasks like approvals, manual interventions, or calling REST APIs.
- Does not support build or deploy tasks.

2. Real Pipeline Example (Multiple Stages)

```

1 trigger:
2   branches:
3     include:
4       - main
5
6 stages:
7   - stage: Build
8     displayName: 'Build Stage'
9     jobs:
10    - job: BuildJob
11      displayName: 'Build and Test'
12      steps:
13        - task: UseDotNet@2
14          inputs:
15            packageType: 'sdk'
16            version: '8.x'
17
18        - script: dotnet build
19          displayName: 'Build Application'
20
21        - script: dotnet test
22          displayName: 'Run Tests'
23
24  - stage: Deploy
25    displayName: 'Deploy Stage'
26    dependsOn: Build
27    condition: succeeded()
28    jobs:
29      - job: DeployJob
30        displayName: 'Deploy to Dev'
31        steps:
32          - script: echo "Deploying application..."
33            displayName: 'Deploy Step'
  
```

6. Steps and Tasks

Step Types



Task
Pre-built task provided by Azure DevOps.



Script
Run custom commands (e.g. bash, PowerShell).



Template
Reuse steps from a template file.

Common Tasks (Examples)

- UseDotNet@2
Install .NET SDK
- DotNetCoreCLI@2
Build, test, publish .NET app
- Docker@2
Build and push Docker image
- AzureCLI@2
Run Azure CLI commands
- PublishPipelineArtifact@1
Publish build artifacts
- DownloadPipelineArtifact@2
Download artifacts

7. Failure Behavior

- ✓ If a step fails, the job stops (by default).
- ✗ If a job fails, the stage is marked as failed.
- ⏸ If a stage fails, the next dependent stages do not run (unless condition: `always()`).
- ⚙ You can control this behavior using conditions.

8. Parallel vs Sequential

Sequential (default)



Stages run one after another.

Parallel (runsAtSameTime)



Stages can run in parallel if there are no dependencies.

9. Key Takeaways

- ✓ Pipeline → Stage → Job → Step → Task/Script.
- ✓ Stages group jobs and control flow.
- ✓ Jobs run on an agent and contain steps.
- ✓ Steps run in order and perform actions.
- ✓ Tasks are pre-built; scripts run custom commands.
- ✓ Use dependencies and conditions to control execution.
- ✓ Understand failure behavior to build reliable pipelines.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Microsoft-Hosted vs Self-Hosted Agents

What an agent actually does, agent pools, hosted runners, self-hosted infrastructure, security implications, and when teams use each.

1. What is an Azure DevOps Agent?



- An agent is a machine that picks up a job from a pipeline and runs the defined steps.
- It can be Microsoft-hosted (provided by Microsoft) or self-hosted (your own machine/VM/server).
- Agents run in an agent pool.

2. How an Agent Works



- 1 Pipeline is triggered.
- 2 An available agent from the selected pool picks up the job.
- 3 The agent runs the defined steps (scripts, tasks, tools).
- 4 It builds, tests, and deploys your application.
- 5 Logs and results are sent back to Azure DevOps.

3. Agent Pools



- An agent pool is a collection of agents.
- You choose an agent pool in your pipeline (pool:).
- Microsoft-hosted agents use the default Microsoft-hosted pool.
- Self-hosted agents require you to create and manage a custom pool.

4. Microsoft-Hosted Agents

Microsoft-Hosted Agents
Provisioned and managed by Microsoft

- ✓ Ready to use (no setup).
- ✓ Multiple OS options (Windows, Ubuntu, macOS).
- ✓ Pre-installed tools (SDKs, Git, Docker, etc.).
- ✓ Automatically updated.
- ✓ Scales automatically.
- ✓ Ideal for most projects and public repositories.

Use When:

- You want a simple, no-maintenance solution.
- Your builds do not need internal network access.
- You use common tools and environments.
- You want automatic scaling and high availability.

5. Self-Hosted Agents

Self-Hosted Agents
Installed and managed by you (on your own infrastructure)

- ✓ Runs on your own machine/VM/server (on-prem or cloud).
- ✓ You install and configure the agent.
- ✓ You control the environment, tools, and network access.
- ✓ Useful for custom tools, internal systems, or specialized hardware.
- ✓ You are responsible for maintenance, updates, and scaling.

Use When:

- You need access to internal resources (private networks, databases, on-prem systems).
- You need custom tools or specific environments.
- You have compliance or security requirements.
- You want to control costs for heavy or long-running jobs.

6. Comparison

Feature	Microsoft-Hosted	Self-Hosted
Provisioning	Ready to use	Manual setup
Maintenance	Managed by Microsoft	Managed by you
Scaling	Automatic	Manual (you scale)
OS Options	Windows, Ubuntu, macOS	Any OS you install
Pre-installed Tools	Yes	You install tools
Network Access	Public internet	Can access internal networks
Security	Shared Microsoft environment	Your own environment (your responsibility)
Cost	Included in Azure DevOps (usage limits apply)	Your infrastructure cost
Best For	Most projects, simple setup	Custom, internal, or compliance needs

7. Security Implications

Microsoft-Hosted

- ✓ Runs in a shared Microsoft environment.
- ✓ You do not have direct control over the machine.
- ✓ Suitable for most scenarios, but not for highly sensitive internal systems.
- ✓ Use secrets and secure variables.
- ✓ Avoid storing sensitive data in the repository.

Self-Hosted

- ✗ Runs in your own environment.
- ✗ You are responsible for securing the machine.
- ✗ Can access your internal network and resources.
- ✗ Ensure regular updates and patching.
- ✗ Restrict network access and use least privilege.
- ✗ Monitor and audit agent usage.

8. Key Takeaways

- ✓ An agent runs your pipeline jobs.
- ✓ Agents are organized in agent pools.
- ✓ Use Microsoft-hosted agents for simple, standard workloads.
- ✓ Use self-hosted agents when you need custom tools, internal access, or special requirements.
- ✓ Understand the security implications of each option.
- ✓ Choose the right agent type based on your project needs, security, and compliance requirements.

Variables, Secrets & Variable Groups

Normal variables vs secrets, variable groups, environment-specific values, and why credentials must never be hardcoded in YAML.

1. What are Pipeline Variables?

- Variables store values used in your pipeline (e.g. app name, environment, paths, URLs).
- They make your pipeline reusable and easier to maintain.
- Variables can be defined in YAML, in the UI, or in variable groups.

Examples:

- appName = my-app
- environment = dev
- buildConfiguration = Release

2. Types of Variables



Normal Variable

- Stores non-sensitive values.
- Visible in logs (unless masked).
- Used for general configuration.



Secret Variable

- Stores sensitive values (e.g. passwords, keys, tokens).
- Masked in logs (shown as ****).
- Use for credentials and secrets.



Variable Group

- A collection of variables (normal and secrets).
- Can be linked to multiple pipelines.
- Useful for environment-specific values (dev, test, prod).

3. Where to Define Variables?



In YAML

Define variables directly in your azure-pipelines.yml.



In Pipeline UI

Add variables in the pipeline settings in Azure DevOps.



In Variable Groups

Create and manage a group of variables and link it to one or more pipelines.



Environment-specific

Use different variable groups for dev, test, and prod environments.

4. YAML Example with Variables

```
variables:
  appName: 'my-app'
  buildConfiguration: 'Release'

stages:
- stage: Build
  jobs:
  - job: BuildJob
    steps:
    - script: echo "Building $(appName)"
      displayName: 'Build Application'
    - script: echo "Configuration: $(buildConfiguration)"
      displayName: 'Show Configuration'
```



5. Using Secrets in YAML

```
variables:
- name: apiKey
  value: $(API_KEY) # from variable group
  isSecret: true

steps:
- script: echo "Using API key..."
  env:
    API_KEY: $(apiKey)
    displayName: 'Call API'
```



- Secrets are masked in logs (shown as ****).
- Do not print secret values.
- Use environment variables (env) to pass secrets to scripts and tools.

6. Variable Groups (Example)

Azure DevOps > Library > Variable groups

Name	Value
appName	my-app
environment	dev
apiUrl	https://dev.api.com
dbUser	**** (secret)
dbPassword	**** (secret)
storageAccount	mydevstorage

- Create variable groups in Azure DevOps.
- Add normal and secret variables.
- Link the group to your pipeline.
- Use different groups for dev, test, prod.

7. Environment-Specific Values

Dev Variable Group	Test Variable Group	Prod Variable Group
- appName = my-app - environment = dev - apiUrl = https://dev.api.com - dbPassword = ****	- appName = my-app - environment = test - apiUrl = https://test.api.com - dbPassword = ****	- appName = my-app - environment = prod - apiUrl = https://api.com - dbPassword = ****



- Use separate variable groups for each environment.
- Link the correct group in your pipeline or release.
- Keep sensitive values (passwords, keys, tokens) as secrets.

8. Why Never Hardcode Secrets?

- Exposes credentials in source code.
- Can be accidentally committed to Git.
- Visible to anyone with repo access.
- Hard to rotate and manage.
- Violates security best practices and compliance requirements.
- Can lead to data breaches and unauthorized access.



Best Practice:

- Store secrets in Azure DevOps variable groups or Key Vault.
- Use secure pipelines and environment approvals.
- Limit access to secrets.
- Rotate secrets regularly.

9. Accessing Variable Group in YAML

```
variables:
- group: my-dev-variables # link variable group

stages:
- stage: Build
  jobs:
  - job: BuildJob
    steps:
    - script: echo "App: $(appName)"
      displayName: 'Show App Name'
    - script: echo "API URL: $(apiUrl)"
      displayName: 'Show API URL'
```

10. Key Takeaways

- Use variables to manage configuration.
- Use secrets for sensitive data.
- Use variable groups for environment-specific values.
- Never hardcode credentials in YAML.
- Follow security best practices and rotate secrets.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta

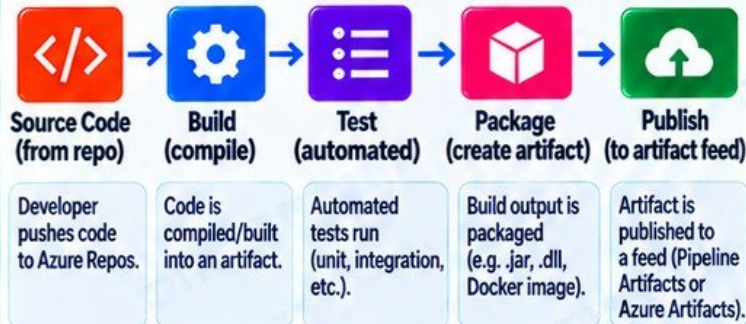


@veriqta

Build, Test & Publish Artifacts

Compile/build → automated tests → package → publish. Explain why deployment should use a known artifact rather than rebuild arbitrary source code.

1. The Build → Test → Publish Flow

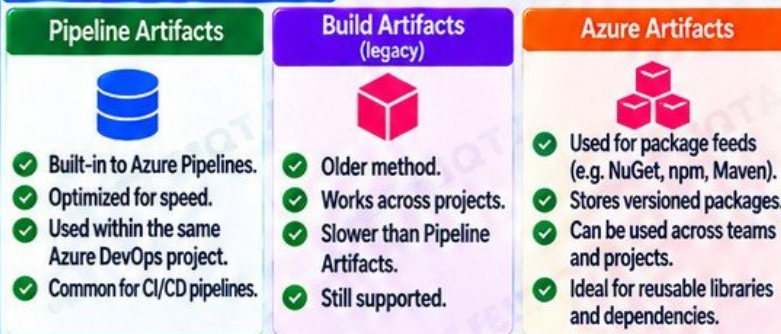


3. What is an Artifact?



- An artifact is the output produced by the build (e.g. compiled code, package, Docker image, etc.).
- It is a fixed, versioned, immutable file that can be used in deployment stages.
- Artifacts are stored in Azure Pipelines or Azure Artifacts.
- Examples: .jar, .dll, .zip, .tar.gz, Docker image, Helm chart.

4. Types of Artifacts



6. Why Use a Known Artifact for Deployment?



- ✓ Ensures the exact same code that was tested is deployed.
- ✓ Provides consistency across environments (dev, test, prod).
- ✓ Avoids build differences due to environment or dependency changes.
- ✓ Improves traceability and rollback (you know what version is running).
- ✗ Rebuilds can produce different results (different dependencies, versions, or build tools).
- ✓ Using a known artifact makes deployments faster, safer, and more reliable.

7. Wrong Approach (Rebuilding in Deployment)



- ✗ Rebuilding source code during deployment can lead to:
- ✗ Different build environment or tool versions.
- ✗ Different dependencies being pulled.
- ✗ Inconsistent artifacts.
- ✗ Harder to troubleshoot issues.
- ✗ Slower deployments.
- ✗ You cannot guarantee you are deploying the same code that was tested.

8. Correct Approach (Use the Published Artifact)

- ✓ Build, test, and publish the artifact in the CI pipeline.
- ✓ Download and deploy the same artifact in CD pipelines.
- ✓ Use versioned artifacts (e.g. build number or git commit).
- ✓ Ensures consistency, security, and traceability.
- ✓ Supports faster and more reliable deployments.

9. Real Pipeline Flow (CI → CD)



10. Key Takeaways



- ✓ Build → Test → Package → Publish.
- ✓ Use pipeline artifacts or Azure Artifacts.
- ✓ Deploy using a known artifact, not by rebuilding source code.
- ✓ Artifacts are versioned, immutable, and traceable.
- ✓ This approach ensures consistent, reliable, and secure deployments.
- ✓ It makes rollbacks and troubleshooting easier.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Azure Artifacts

Feeds and packages, including NuGet, npm, Maven, Python packages, and Universal Packages.
Explain where package management fits into CI/CD.

1. What is Azure Artifacts?

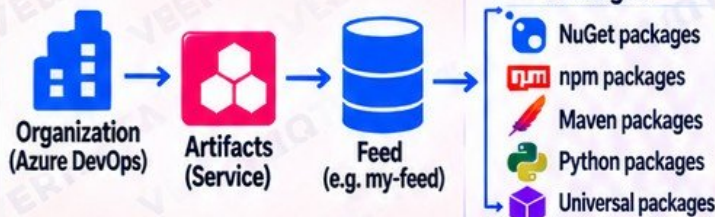


- ✓ Azure Artifacts is a package management service in Azure DevOps.
- ✓ It allows you to create, host, and share packages within your organization.
- ✓ Supports multiple package types: NuGet, npm, Maven, Python (PyPI), and Universal Packages.
- ✓ Helps teams reuse code, manage dependencies, and keep packages secure and private.

2. Supported Package Types

NuGet (.NET)	npm (Node.js)	Maven (Java)	Python (PyPI)	Universal Packages
				
- Used for (.NET libraries and tools. - Packages have .nupkg extension.	- Used for JavaScript/Node.js packages. - Packages stored from package.json.	- Used for Java libraries and apps. - Packages have .jar extension.	- Used for Python packages. - Packages stored from setup.py or pyproject.toml.	- Used for any type of file (e.g. scripts, binaries, Docker files, Helm charts). - Great for custom packages.

3. Azure Artifacts Feed



- ✓ A feed is a collection of packages.
- ✓ Feeds can be scoped to a project or the entire organization.
- ✓ You can control access with permissions (read, write, manage).

4. Where Azure Artifacts Fits in CI/CD



- ✓ During CI, your build creates a package (e.g. .nupkg, .tgz, .jar, .whl).
- ✓ The package is published to an Azure Artifacts feed.
- ✓ During CD or in other projects, the package is downloaded from the feed and used for deployment.
- ✓ This ensures you deploy a known, tested, and versioned package.

5. Example: Publish and Use Packages

NuGet (.NET)	npm (Node.js)	Maven (Java)	Python (PyPI)	Universal Packages
# Publish <pre>dotnet nuget push \ **/*.nupkg \ --source "\$(FeedUrl)" \ --api-key "\$(System.AccessToken)"</pre> # Restore (use package) <pre>dotnet restore \ --source "\$(FeedUrl)"</pre>	# Publish <pre>npm config set \ registry \$(FeedUrl) \ npm publish</pre> # Install (use package) <pre>npm install \ --registry \$(FeedUrl)</pre>	# settings.xml (snippet) <pre><repositories> <repository> <id>azure-artifacts</id> <url>\$(FeedUrl)</url> </repository> </repositories></pre> # Publish <pre>mvn deploy</pre>	# Publish <pre>pip install twine twine upload \ --repository-url \$(FeedUrl) \ dist/*</pre> # Install (use package) <pre>pip install \ --index-url \$(FeedUrl) \ my-package</pre>	# Publish <pre>az artifacts universal publish \ --organization \$(Org) \ --project \$(Project) \ --scope project \ --feed my-feed \ --name my-package \ --version 1.0.0 \ --path ./files</pre> # Download (use package) <pre>az artifacts universal download \ --organization \$(Org) \ --project \$(Project) \ --scope project \ --feed my-feed \ --name my-package \ --version 1.0.0 \ --path ./download</pre>

6. Benefits



- ✓ Centralized package storage.
- ✓ Reuse code across teams and projects.
- ✓ Supports multiple package types.
- ✓ Versioning and traceability.
- ✓ Access control and security.
- ✓ Integrates with Azure Pipelines.
- ✓ Reduces build time and external dependency risks.

7. Best Practices



- ✓ Use private feeds for internal packages.
- ✓ Set proper permissions (least privilege).
- ✓ Use versioning (semantic versioning).
- ✓ Keep sensitive packages secured.
- ✓ Clean up old versions regularly.
- ✓ Use universal packages for non-standard files.
- ✓ Integrate with CI/CD pipelines.

8. Key Takeaways



- ✓ Azure Artifacts manages and stores packages.
- ✓ Supports NuGet, npm, Maven, Python, and Universal Packages.
- ✓ Feeds organize packages and control access.
- ✓ Package management is a key part of CI/CD.
- ✓ Deploy from a known, tested, and versioned package, not from arbitrary source code.
- ✓ Improves security, consistency, and reliability across environments.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

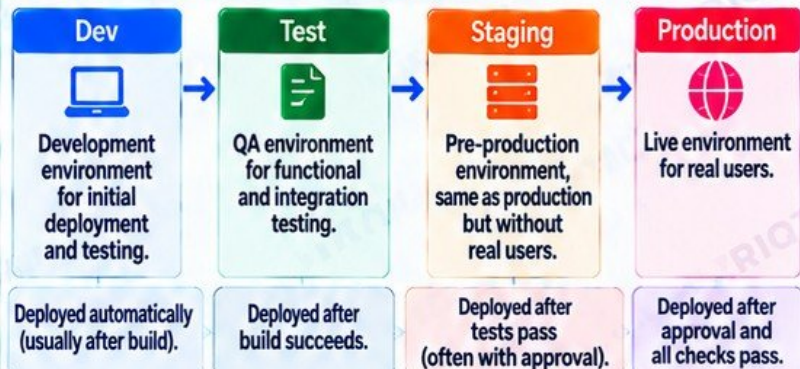
Environments & Deployment Stages

Dev → Test → Staging → Production. Show how the same application artifact progresses safely through environments.

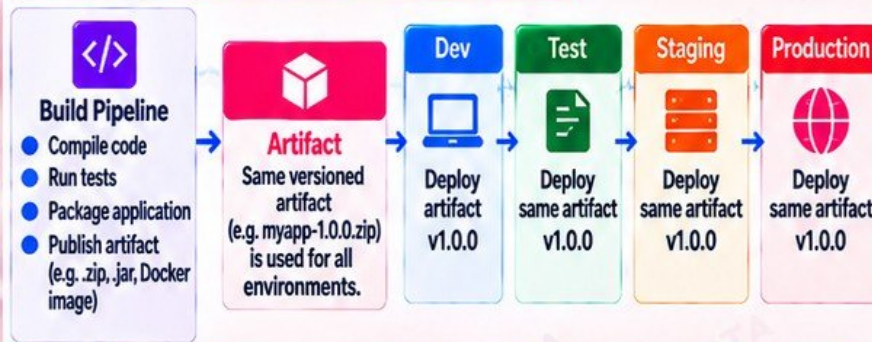
1. What are Environments?

- ✓ Environments represent the target infrastructure for your application (e.g. Dev, Test, Staging, Production).
- ✓ They allow safe, controlled deployments with approvals, checks, and security.
- ✓ Each environment can have different configuration, variables, and access control.
- ✓ Used in Azure DevOps Release Pipelines or Multi-stage YAML pipelines.

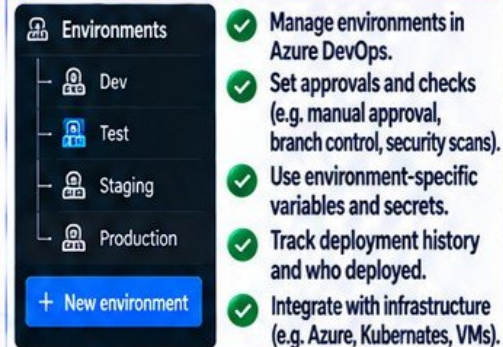
2. Typical Deployment Stages



3. How the Same Artifact Progresses



4. Azure DevOps Environments



5. Example Multi-Stage Pipeline (YAML)

```

stages:
- stage: Dev
  displayName: 'Deploy to Dev'
  jobs:
  - deployment: DeployDev
    environment: dev
    strategy:
      runOnce:
        deploy:
          steps:
          - script: echo "Deploying to Dev"
- stage: Test
  displayName: 'Deploy to Test'
  dependsOn: Dev
  jobs:
  - deployment: DeployTest
    environment: test
    strategy:
      runOnce:
        deploy:
          steps:
          - script: echo "Deploying to Test"
- stage: Staging
  displayName: 'Deploy to Staging'
  dependsOn: Test
  jobs:
  - deployment: DeployStaging
    environment: staging
    strategy:
      runOnce:
        deploy:
          steps:
          - script: echo "Deploying to Staging"
- stage: Production
  displayName: 'Deploy to Production'
  dependsOn: Staging
  jobs:
  - deployment: DeployProd
    environment: production
    strategy:
      runOnce:
        deploy:
          steps:
          - script: echo "Deploying to Production"
  
```

6. Environment-Specific Configuration

Setting	Dev	Test	Staging	Production
API URL	https://dev.api.com	https://test.api.com	https://staging.api.com	https://api.com
Database	dev-db	test-db	staging-db	prod-db
Log Level	Debug	Info	Info	Warning
Feature Flags	Enabled	Enabled	Limited	Limited
Scale	1 instance	2 instances	2-4 instances	Auto-scale

7. Approvals and Checks



8. Why Use the Same Artifact?

- ✓ Ensures the exact same code that was tested is deployed to all environments.
- ✓ Avoids "it works in test but not in production" issues.
- ✓ Improves consistency, reliability, and traceability.
- ✓ Prevents accidental changes between environments.
- ✓ Makes rollbacks easier (same versioned artifact).
- ✓ Supports auditing and compliance requirements.

9. Deployment Flow with Checks



10. Key Takeaways

- ✓ Use multiple environments (Dev, Test, Staging, Production).
- ✓ Deploy the same versioned artifact across all environments.
- ✓ Use approvals and checks for higher environments.
- ✓ Configure environment-specific variables and secrets.
- ✓ Track deployments and maintain audit history.
- ✓ This approach ensures safe, reliable, and consistent deployments.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Approvals, Checks & Deployment Safety

Production approvals, checks, protected resources, verification, rollback thinking, and why a successful pipeline does not automatically mean a healthy application.

1. What are Approvals and Checks?



- ✓ Approvals and checks control deployments to protected environments (e.g. staging, production).
- ✓ They ensure the right people, conditions and validations are met before deployment.
- ✓ They reduce the risk of accidental or unauthorized changes.
- ✓ You can add multiple checks (approvals, branch policies, security scans, etc.).

2. Deployment with Approvals and Checks (Flow)



3. Types of Approvals

Manual Approval



- A human must approve the deployment.
- Used for production or sensitive environments.
- You can require multiple approvers.

Business Hours



- ✓ Allow deployments only during specific time windows.
- ✓ Example: 9 AM – 5 PM (Monday – Friday).
- ✓ Prevents risky changes at night or weekends.

Branch Control



- Allow deployments only from specific branches (e.g. main).
- ✓ Prevents deploying unreviewed code.
- ✓ Works with branch policies in Azure Repos.

4. Common Checks in Azure DevOps

✓ Approvals	Manual approval by authorized users.
⚙️ Branch control	Deploy only from specific branches.
🔒 Required template	Enforce use of a specific YAML template.
🔍 Security scan	Run security tools (e.g. SAST, dependency scan).
🕒 Business hours	Allow deployments only in defined time windows.
🌐 Invoke REST API	Call external systems for validation.
✓ Azure Function	Run custom validation logic.
👤 Check for work items	Ensure required work items are completed.

5. Protected Resources



Environments (Dev, Test, Prod)



Kubernetes Clusters



Azure Subscriptions



Service Connections

- ✓ You can add checks and approvals to these resources.
- ✓ Helps prevent unauthorized or accidental deployments.
- ✓ Supports compliance and audit requirements.
- ✓ Limits who can deploy and under what conditions.

6. Verification After Deployment



- ✓ Check application health (endpoints, logs, metrics).
- ✓ Verify key functionality (smoke tests).
- ✓ Monitor error rates, latency and traffic.
- ✓ Validate that the correct version is running.
- ✓ Confirm database migrations (if any) completed successfully.

7. Rollback Thinking



- ✗ Always have a rollback plan before deploying.
- ✗ Use versioned artifacts to quickly roll back.
- ✗ Keep previous versions available in Azure Artifacts or container registry.
- ✗ Test rollback procedures regularly.
- ✗ Rollback is faster and safer when you deploy a known artifact.

8. Why a Successful Pipeline ≠ Healthy Application?

✓ Pipeline Success Means

Code compiled successfully
Tests passed
Package was created
Deployment completed
No errors in pipeline logs

✗ Application Can Still Have Issues

Runtime errors
Configuration problems
Database issues
Performance problems
Dependency failures
External service issues
Users cannot access key features

9. Deployment Safety Checklist



- ✓ Use approvals for production.
- ✓ Add security checks and scans.
- ✓ Deploy only from trusted branches.
- ✓ Validate after deployment (health checks, smoke tests).
- ✓ Monitor the application and set up alerts.
- ✓ Have a rollback plan and test it.
- ✓ Use the same versioned artifact across all environments.
- ✓ Keep an audit trail of who approved and what was deployed.

10. Key Takeaways



- ✓ Approvals and checks prevent risky deployments.
- ✓ Protect environments and critical resources.
- ✓ A successful pipeline does not guarantee a healthy application.

- ✓ Always verify after deployment.
- ✓ Have a rollback plan.
- ✓ Use versioned artifacts for consistency and faster rollbacks.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Service Connections & Azure Authentication

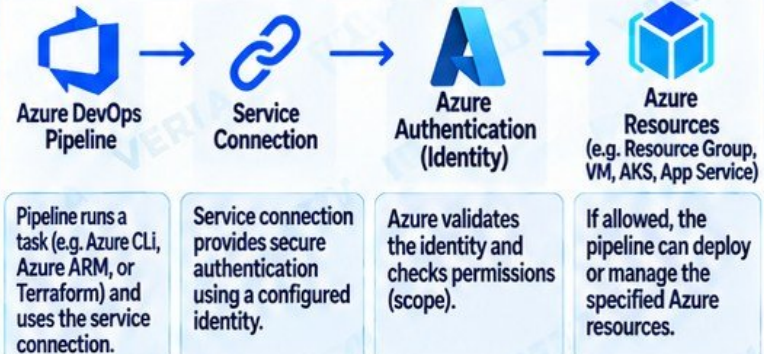
How Azure DevOps receives permission to deploy Azure resources. Explain service connections, identities, scope, least privilege, and credential risk.

1. What are Service Connections?



- ✓ A service connection is a secure connection in Azure DevOps to an external service (e.g. Azure, AWS, Docker, Kubernetes).
- ✓ It stores the authentication details that pipelines use to access resources.
- ✓ It allows pipelines to perform actions without storing long-lived credentials in your YAML code.
- ✓ For Azure, it tells Azure DevOps which identity to use and what resources it can access.

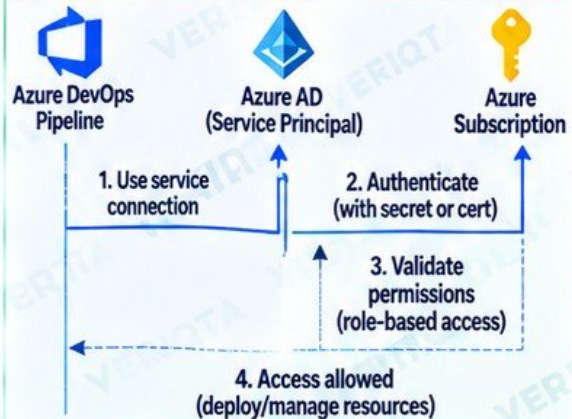
2. How It Works (High Level)



3. Types of Azure Service Connections

Service Principal (Client Secret)	Service Principal (Certificate)	Managed Identity (System-Assigned or User-Assigned)	Workload Identity (Federation)
			
✓ Uses an Azure AD service principal with a client secret. ✓ Common and widely used. ✓ Secret must be stored securely.	✓ Uses a service principal with a certificate. ✓ More secure than client secret. ✓ Recommended for production in many cases.	✓ Uses a managed identity from Azure e.g. VM, App Service, AKS, or self-hosted agent. ✓ No secrets to rotate. ✓ Very secure and recommended when possible.	✓ Uses OIDC federation (no client secret). ✓ Recommended for modern pipelines (e.g. GitHub, Azure DevOps, or external CI/CD). ✓ More secure and eliminates long-lived credentials.

4. Authentication Flow (Service Principal)



5. Scope and Permissions



- ✓ A service connection has a scope (subscription, resource group, or specific resource).
- ✓ Use the minimum scope required (least privilege).
- ✓ Assign only the roles needed (e.g. Contributor, Reader, or custom role).
- ✓ Avoid using Owner unless absolutely necessary.

6. Example: Create an Azure Service Connection

- 1 Create an Azure AD service principal.
- 2 Assign required role (e.g. Contributor) to a resource group.
- 3 In Azure DevOps, go to Project Settings → Service connections.
- 4 Click New service connection → Azure Resource Manager.
- 5 Choose authentication method (secret, certificate or workload identity).
- 6 Provide subscription, service principal details and test the connection.



Azure DevOps Service Connection

+ New service connection

Connection type: Azure Resource Manager

Authentication method: Service principal (client secret)

Scope level: Subscription

Subscription: <your-subscription>

Verify and save

7. Using the Service Connection in YAML

```
- task: AzureCLI@2
  inputs:
    azureSubscription: 'My-Azure-ServiceConnection' # service connection name
    scriptType: 'bash'
    scriptLocation: 'inlineScript'
    inlineScript: |
      az account show
      az group list --output table
```



8. Credential Risk and Security Best Practices



- ✗ Do not hardcode credentials in YAML.
- ✗ Avoid storing secrets in pipeline variables.
- ✗ Limit the scope and use least privilege.
- ✗ Use managed identity or workload identity when supported.
- ✗ Rotate credentials regularly (if using secrets/certificates).
- ✓ Use Azure DevOps secure service connections.
- ✓ Enable auditing and monitoring.
- ✓ Review permissions periodically.
- ✓ Use separate service connections for dev, test and prod.
- ✓ Follow your organization's security and compliance requirements.

9. Common Use Cases

					
Deploy ARM Templates	Deploy to Azure App Service	Deploy to AKS (Kubernetes)	Manage Resource Groups	Use with Terraform/CLI	Access Azure Key Vault

10. Key Takeaways



- ✓ Service connections securely authenticate your pipeline to Azure.
- ✓ Choose the right identity type for your needs.
- ✓ Use the minimum required scope and permissions.
- ✓ Never hardcode credentials in YAML.
- ✓ Monitor, audit and rotate credentials regularly.
- ✓ Use managed identity or workload identity when possible.
- ✓ Proper authentication and least privilege reduce security risk.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Infrastructure as Code in Pipelines

Where Bicep and Terraform fit. Code → Validate/Plan → Review → Apply → Verify.
Introduce infrastructure deployment without turning the page into a Terraform course.

1. What is Infrastructure as Code (IaC)?

- ✓ Infrastructure as Code (IaC) lets you define cloud infrastructure using code (e.g. Bicep, Terraform) instead of manual clicks.
- ✓ The same code is versioned, reviewed, tested, and deployed through Azure Pipelines.
- ✓ It makes infrastructure repeatable, consistent, and easier to manage.
- ✓ Changes are tracked in Git just like application code.

2. Where Bicep and Terraform Fit

Bicep (Azure)



- ✓ Microsoft's IaC language for Azure.
- ✓ Deploys Azure resources (e.g. VMs, AKS, Storage).
- ✓ Simpler syntax, native Azure integration.
- ✓ Best when you are primarily building on Azure.

Terraform (Multi-Cloud)



- ✓ Open-source IaC tool by HashiCorp.
- ✓ Works with Azure, AWS, GCP and many providers.
- ✓ Uses a provider model.
- ✓ Best when you need multi-cloud or existing Terraform skills.

3. IaC Deployment Flow in a Pipeline



4. Example Pipeline (Bicep)

```
# azure-pipelines.yml (Bicep example)
trigger:
- main
pool:
vmImage: 'ubuntu-latest'
stages:
- stage: Deploy_Infrastructure
  displayName: 'Deploy Azure Infrastructure (Bicep)'
  jobs:
  - job: deploy
    displayName: 'Validate, Deploy and Verify'
    steps:
    - task: AzureCLI@2
      inputs:
        azureSubscription: 'My-Azure-ServiceConnection'
        scriptType: 'bash'
        scriptLocation: 'inlineScript'
        inlineScript: |
          az deployment group create \
            --resource-group my-rg \
            --template-file infra/main.bicep \
            --parameters @infra/parameters.prod.json
```

5. Example Pipeline (Terraform)

```
# azure-pipelines.yml (Terraform example)
trigger:
- main
pool:
vmImage: 'ubuntu-latest'
stages:
- stage: Terraform_Deploy
  jobs:
  - job: plan_and_apply
    steps:
    - script: terraform init
      displayName: 'Terraform Init'
    - script: terraform plan -out=tfplan
      displayName: 'Terraform Plan'
    - script: terraform apply -auto-approve tfplan
      displayName: 'Terraform Apply'
```

6. What the Plan Shows



- ✓ A plan shows the exact infrastructure changes that will be made.
- ✓ You can review the plan before applying.
- ✓ Helps catch mistakes and unwanted changes.
- ✓ Use approvals for production environments.

7. Best Practices



- ✓ Keep IaC code in Git (version control).
- ✓ Always run validate/plan before apply.
- ✓ Use approvals for staging and production.
- ✓ Use separate state files (Terraform).
- ✓ Use service connections with least privilege access.
- ✓ Keep environment-specific parameters (e.g. dev, test, prod).
- ✓ Review plans for unexpected changes.
- ✓ Enable logging and monitoring.

8. Common Azure Resources Managed with IaC



9. Benefits of IaC in Pipelines



- ✓ Repeatable and consistent environments.
- ✓ Faster and safer deployments.
- ✓ Full audit trail (who changed what).
- ✓ Easier rollback using version control.
- ✓ Reduces configuration drift.
- ✓ Infrastructure and application can be deployed together.

10. Key Takeaways



- ✓ IaC (Bicep/Terraform) defines your cloud infrastructure using code.
- ✓ Use validate/plan → review → apply → verify in your pipeline.
- ✓ Keep code in Git, use approvals, and follow least privilege.
- ✓ Bicep is great for Azure; Terraform is good for multi-cloud.
- ✓ A plan helps you see changes before deployment.
- ✓ IaC in pipelines makes infrastructure deployments safer, faster, and reliable.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Containers & AKS Delivery

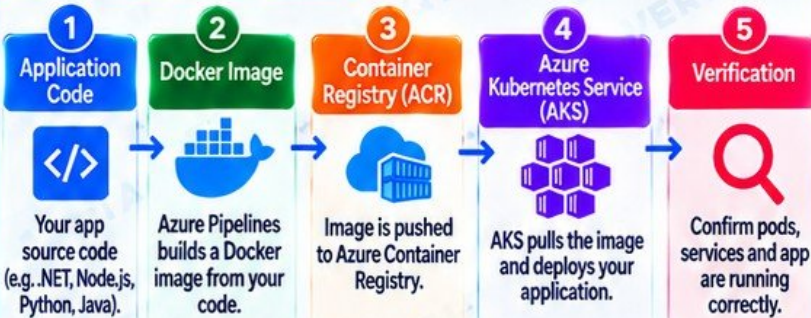
App → Docker image → Container Registry → Azure Kubernetes Service → verification.
Show where Azure Pipelines participates in container delivery.

1. What is Container Delivery?

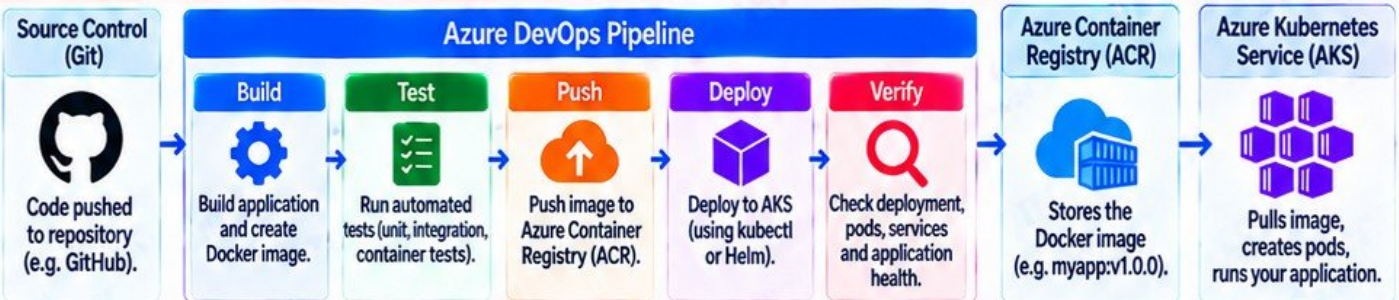


- ✓ You package your application into a Docker image.
- ✓ The image is stored in a container registry (e.g. Azure Container Registry).
- ✓ Azure Kubernetes Service (AKS) pulls the image and runs it as containers.
- ✓ Azure Pipelines automates this entire process from build to deployment.

2. High-Level Flow



3. End-to-End Architecture with Azure Pipelines



4. Example Pipeline (Build, Push and Deploy to AKS)

```
# azure-pipelines.yml (simplified example)
trigger:
- main
variables:
  imageName: 'myapp'
  acrName: 'myacr'
  repository: '$(acrName).azurecr.io/$(imageName)'
stages:
- stage: Build_And_Push
  jobs:
  - job: Build
    steps:
    - task: Docker@2
      displayName: 'Build and Push Docker Image'
      inputs:
        containerRegistry: 'my-acr-service-connection'
        repository: '$(repository)'
        command: 'buildAndPush'
        Dockerfile: '**/Dockerfile'
        tags: '$(Build.BuildId)'
- stage: Deploy_To_AKS
  dependsOn: Build_And_Push
  jobs:
  - job: Deploy
    steps:
    - task: Kubernetes@1
      displayName: 'Deploy to AKS'
      inputs:
        connectionType: 'Azure Resource Manager'
        azureSubscriptionEndpoint: 'my-azure-service-connection'
        kubernetesCluster: 'my-aks-cluster'
        namespace: 'myapp'
        command: 'apply'
        useConfigurationFile: true
        configuration: 'k8s/deployment.yaml'
```

YAML

5. Key Components

	Application Code	Your application source code (e.g. .NET, Node.js, Python, Java).
	Docker Image	A lightweight, portable image that contains your application and its dependencies.
	Azure Container Registry (ACR)	Private registry to store Docker images in Azure.
	Azure Kubernetes Service (AKS)	Managed Kubernetes service to run your containerized application.
	Azure Pipelines	Automates build, test, push and deployment to AKS.

6. Verification Steps

- ✓ Check pods: kubectl get pods
- ✓ Check services: kubectl get svc
- ✓ Check application logs: kubectl logs <pod>
- ✓ Test application endpoint (curl or browser).
- ✓ Verify the correct image version is running.
- ✓ Monitor health, metrics and alerts.

7. Best Practices

- ✓ Use a private container registry (ACR).
- ✓ Scan images for vulnerabilities (e.g. with Trivy or Azure Defender).
- ✓ Use version tags (not latest).
- ✓ Store secrets in Azure Key Vault or variable groups (not in code).
- ✓ Use Kubernetes manifests or Helm charts for deployment.
- ✓ Use approvals for production deployments.
- ✓ Implement health checks and monitoring.
- ✓ Keep images small and secure.

8. Common Tools Used



9. Key Takeaways

- ✓ Azure Pipelines builds, tests and pushes Docker images to ACR.
- ✓ AKS pulls the image and runs your application.
- ✓ Always verify the deployment after release.
- ✓ Use security best practices (scanning, secrets, approvals).
- ✓ This creates a repeatable, reliable and scalable container delivery process.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta

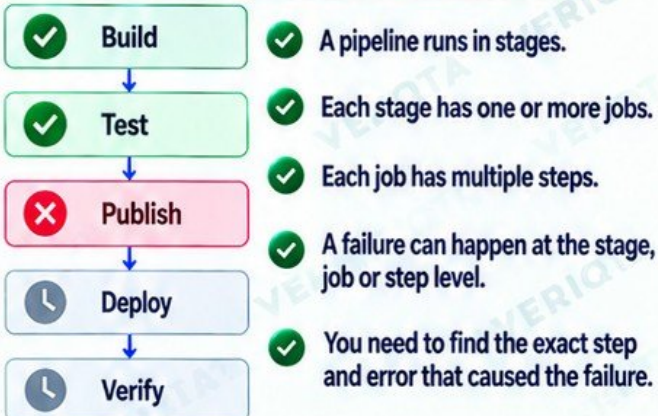


@veriqta

When the Pipeline Fails

A junior troubleshooting workflow: identify failed stage → inspect job → read logs → identify first meaningful error → check variables/permissions/artifacts → fix → rerun → verify.

1. Understand the Pipeline Structure



2. Step 1 – Identify the Failed Stage

Jobs in run #20251010.1
MyApp-CI-CD

✓ Build	1m 12s
✓ Test	2m 34s
✗ Publish	32s
⌚ Deploy	<1s
⌚ Verify	<1s

What to Do

- ✓ Open the pipeline run.
- ✓ Find the failed stage (red X).
- ✓ Click on the stage to see the jobs inside.
- ✓ Identify which job failed.
- ✓ Click the job to view the steps and logs.

3. Step 2 – Inspect the Job and Steps

✗ Publish	32s
✓ Initialize job	2s
✓ Download artifact	3s
✗ Push to container registry	25s
⌚ Post-job: clean up	2s

What to Do

- ✓ Look at all steps in the job.
- ✓ Find the failed step (red X).
- ✓ Click the step to open the logs.
- ✓ Check the exact command that failed.
- ✓ Don't just look at the last line find the first meaningful error.

4. Step 3 – Read the Logs

✗ Push to container registry

```
12 # Pushing image to ACR
13 docker push myapp.azurecr.io/myapp:latest
14 The push refers to repository
15 denied: requested access to the resource
16 is denied
17 ERROR: failed to push image
18 ##[error]Docker command failed with exit code 1
```

What to Look For

- ✓ Find the first real error message (not all the noise after it).
- ✓ Look for keywords: error, denied, not found, auth, permission, failed.
- ✓ Read a few lines before the error to understand the context.

5. Step 4 – Identify the Root Cause

Variables	Permissions	Artifacts	Dependencies
			
Are required variables set? Are secrets available?	Does the service connection or identity have the right permissions?	Is the artifact created and available?	Are external services (ACR, AKS, Azure, etc.) reachable?

6. Step 5 – Fix the Issue



- ✓ Update the code, YAML, variables or permissions based on the root cause.
- ✓ Test the fix locally if possible (e.g. docker build, terraform plan).
- ✓ Commit and push your changes to Git (version control).
- ✓ Or update variables, service connection or permissions in Azure DevOps.

7. Step 6 – Rerun the Pipeline



Run pipeline

- ✓ Rerun the failed pipeline.
- ✓ You can rerun the entire pipeline or just the failed stage (when supported).
- ✓ Make sure the changes you made are included.
- ✓ Check that the pipeline completes successfully.

8. Step 7 – Verify the Fix



- ✓ Confirm all stages pass (green ✓).
- ✓ Verify the application is deployed and working.
- ✓ Check the deployed environment (e.g. AKS, App Service, etc.).
- ✓ Test key functionality (endpoints, UI, logs, metrics).
- ✓ Make sure there are no hidden issues.

9. Common Failure Causes

- ✗ Missing or incorrect variables.
- ✗ Expired or wrong credentials.
- ✗ Insufficient permissions.
- ✗ Artifact not found.
- ✗ Docker build or push failures.
- ✗ Network or service connectivity issues.
- ✗ Invalid YAML syntax or task configuration.
- ✗ Resource limits (quota, memory, etc.).

10. Key Takeaways



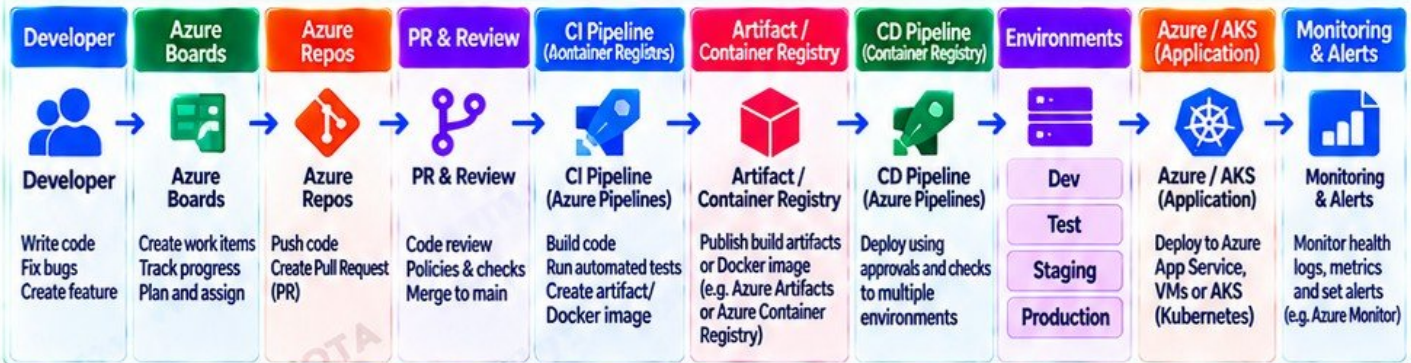
- ✓ Always find the first meaningful error.
- ✓ Check variables, permissions and artifacts.
- ✓ Fix the root cause, not just the symptom.

- ✓ Use logs, don't guess.
- ✓ Rerun and verify in the actual environment.
- ✓ Troubleshooting is a key skill for every DevOps engineer.

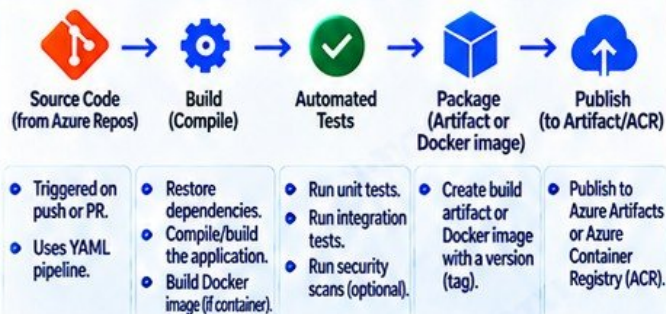
Production CI/CD Architecture

Bringing everything together: Developer → Azure Boards → Azure Repos → PR → CI Pipeline → Tests → Artifact/Container Registry → CD Pipeline → Dev/Test/Prod → Azure/AKS → Monitoring.

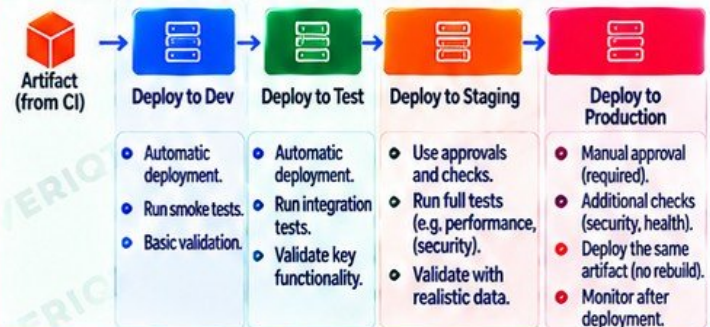
1. End-to-End CI/CD Architecture (High Level)



2. CI Pipeline (Build, Test and Package)



3. CD Pipeline (Deploy to Multiple Environments)



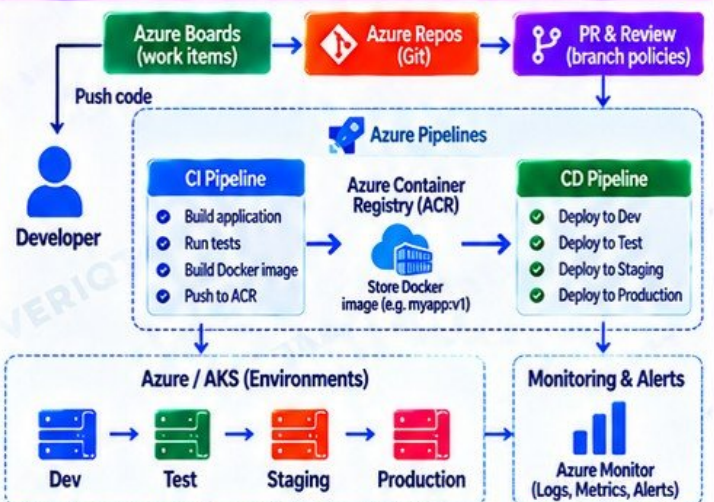
4. Where Azure Services Fit

	Azure Boards	Track work items, plan sprints, manage tasks.
	Azure Repos	Store source code (Git) and manage PRs.
	Azure Pipelines	Build, test and deploy (CI/CD).
	Azure Artifacts	Store build artifacts and packages (e.g. NuGet, npm).
	Azure Container Registry	Store Docker container images.
	Azure Kubernetes Service (AKS)	Run containerized applications.
	Azure App Service / VMs	Alternative deployment targets.
	Azure Monitor	Monitor logs, metrics, alerts and application health.

6. Key Practices for a Production-Ready Pipeline

- ✓ Use the same build artifact or container image across all environments.
- ✓ Use approvals and checks for staging and production.
- ✓ Keep credentials in service connections and variable groups (not in code).
- ✓ Run automated tests (unit, integration, security).
- ✓ Scan container images for vulnerabilities.
- ✓ Use infrastructure as code (Bicep or Terraform) for environment setup.
- ✓ Monitor the application after deployment.
- ✓ Have a rollback plan and test it.
- ✓ Keep your pipeline YAML in source control.
- ✓ Follow least privilege access and secure your pipeline.

5. Example CI/CD Architecture (Azure & AKS)



7. Common Tools and Technologies



8. Benefits of This Architecture

- ✓ Automates the entire delivery process.
- ✓ Faster and more reliable releases.
- ✓ Consistent and repeatable deployments.
- ✓ Better collaboration and tracking.
- ✓ Improved security and compliance.
- ✓ Easier troubleshooting and rollback.

9. Real-World Example Flow

- Developer creates a feature and pushes code to Azure Repos.
- PR is created, reviewed and merged.
- CI pipeline builds, tests and publishes a Docker image to ACR.
- CD pipeline deploys to Dev → Test → Staging → Production.
- Application runs on AKS and is monitored with Azure Monitor.

10. Key Takeaways

- ✓ End-to-end CI/CD connects people, code, infrastructure and monitoring.
- ✓ Use Azure DevOps for automation and control.
- ✓ Deploy the same tested artifact to all environments.
- ✓ Use approvals, checks and monitoring for safety.
- ✓ This architecture works for both simple apps and complex enterprise systems.